Метрики здоровья репозитория

Версия rhs-2.0-rc1 · 16 сентября 2026 года

Текущий кандидат методики: точные формулы, состав показателей, ограничения и контрольные примеры. Эталонный расчёт на Go проверен на синтетических данных. Параметры ещё не откалиброваны на реальных репозиториях.

Работающий MVP пока использует mvp-1.0. Этот документ описывает следующую версию расчёта; он не означает, что все источники данных уже подключены.

Публичная и личная оценки

Категория	public	personal
documentation	20	16
ci	25	20
code	25	20
activity	15	12
issues	15	12
security	не входит	20

Сумма каждого профиля - 100. Публичный профиль доступен только для подтверждённо публичного репозитория. В нём Security не является отсутствующей категорией: она не входит в состав вообще. Агрегатор полностью игнорирует личные security-входы, включая ошибочные и исторические, и не возвращает их в публичной проекции. Остальные входы публичного профиля должны быть собраны исключительно из публичных данных.

Личный профиль всегда отмечен как личный; он никогда не получает место в публичном рейтинге. Даже публичные категории личного репозитория могут быть закрытыми. Проверка пользователей, выдача отчётов, ключи кешей и повторная проверка прав - обязанность сервисов доступа и сбора, а не арифметической библиотеки.

Принцип расчёта

Взвешенное среднее по измеренным фактам, отдельная полнота данных и ограничение итогового личного балла при критической находке AppSec. Отсутствие данных не превращается в плохой результат.

01 / Данные и состояния

Расчёт не обращается к сети, SQL, файловой системе и системному времени. Вход содержит `profile`, явные `at` и `as_of` в UTC, текущий подтверждённый `public_repository`, пять нормализованных категорий, а в личном профиле также статус AppSec, находки и историческое наличие ограничения.

`at` - время завершения сбора совместимого снимка; `as_of` - время оценки. Требуется `at <= as_of`. Время ревизии, запуска проверки и получения ответа API - разные поля нормализованных фактов, которые адаптер обязан учитывать до создания снимка.

Для каждого исходного измерения сборщик хранит: идентификатор репозитория, ревизию, идентификатор источника/метода, период наблюдения, время получения, полноту пагинации, контекст доступа, единицы и ссылки на доказательства. Ответы API и токены не передаются в аналитический отчёт целиком. Соответствие этим условиям не доказывается одним успешным вызовом арифметической функции.

Эталонный агрегатор принимает уже рассчитанные `Value`: `state`, nullable `score` в 0-100, `coverage` в 0-1. Категории получают значения через функции этого пакета. Прямое введение готовых оценок допустимо в синтетических сценариях. Клиент браузера не должен иметь возможности прислать свои баллы как результаты анализа.

Состояния:

State	Значение
known	Измеренное значение; score конечен и в 0-100, coverage > 0 и <= 1
missing	Источник или необходимые сведения отсутствуют
denied	Нет разрешения читать источник
error	Ошибка получения или преобразования
incomplete	Неполная пагинация, усечённый код или незавершённый состав проверок
stale	Сведения не относятся к требуемому актуальному состоянию
insufficient	Слишком мало наблюдений по правилам конкретной метрики
not_applicable	Неприменимо к данному проекту по установленным правилам

У всех неизвестных состояний `score=null`, `coverage=0`. Они не становятся нулём. В фиксированном профиле rc1 даже not_applicable не уменьшает ожидаемый знаменатель полноты. Разные типы проектов с отсутствующими категориями не ранжируются как полные; специальные профили требуют отдельной версии.

Отрицательные счётчики, good > total, неизвестные enum, NaN/Infinity, противоречащие друг другу дубликаты - ошибки входа. Адаптер переводит ошибку соответствующего источника в `error`, не публикует расчёт по повреждённым данным и не подставляет 0/100.

02 / Документация

Функция: `Documentation([10]*bool)`. true = найдено доказательство; false = после полного разрешённого чтения подтверждено отсутствие; nil = неизвестно. Счёт подпункта 100/0, неизвестный вклад исключается только из числителя измеренных весов.

Индексы	Проверяемые сведения	Вес каждого
0, 1	Назначение; пример использования или описание интерфейса	10, 10
2, 3	Требования/зависимости; последовательность установки, запуска или подключения	15, 15
4, 5	Способ сборки; способ запуска тестов	10, 10
6, 7	Канал обращения/контакт; порядок предложения изменений	7,5; 7,5
8, 9	Явное обозначение лицензии; её текст или ссылка на текст	7,5; 7,5

Функция не анализирует текст автоматически. Нормализатор обязан предъявлять путь и фрагмент, который обосновывает true. Одного заголовка, имени файла, размера текста или универсального шаблона недостаточно для подтверждения конкретного пункта. В rc1 допустима ручная аннотация контрольных документов по этой рубрике; автоматический детектор ещё не подключён и должен иметь отдельные проверяемые правила. Корректность команд не проверяется выполнением; юридическая применимость лицензии не оценивается.

Результат - полнота заявленных сведений, не гарантия их достоверности. Например, один неизвестный пункт веса 15 при остальных true даёт score=100, coverage=0,85; тот же пункт с подтверждённым false даёт score=85, coverage=1.

03 / CI/CD

Функция `CI(success,total)`: `100 * success / total`, минимум 5 разных подходящих единиц наблюдения. Меньшая выборка -> insufficient.

Период: первые завершённые попытки, закончившиеся в `[at-30 суток, at]`. Основная ветка на момент снимка. Список значимых pipeline задаётся адаптером из подтверждённой конфигурации, фиксируется в доказательствах. Для каждой пары `(revision, pipeline_id)` выбирается первая терминальная попытка по `(finished_at, run_id)`. Один и тот же запуск не учитывается повторно. Список не расширяется автоматически случайными вспомогательными заданиями.

success входит в числитель; success, failed, timeout - в знаменатель. pending/running/cancelled/skipped исключаются, их количества сохраняются справочно. Нельзя считать несуществующий результат отменённого запуска успехом. Если в истории отсутствует первая попытка либо выборка усечена, метрика incomplete.

Минимум 5 единиц дополнительно требует 5 различных ревизий; это условие проверяет адаптер до вызова CI. Наличие файла CI не даёт бонуса. Отсутствие доступных запусков не доказывает отсутствие внешнего CI и не даёт ноль.

Текущий статус основной ветки, успех после повторной попытки и длительность отображаются справочно. В rc1 отдельного ограничения общего Score по CI нет: до проверки контракта API условие обязательной сборки не определено достаточно надёжно.

Пример

Четыре успешные первые попытки из пяти подходящих ревизий: CI = 80. Один успешный запуск: недостаточно наблюдений, а не оценка 100 за надёжность.

04 / Issues

`Response(onTime,total)` = `100 * onTime / total`, минимум 5 наблюдений. Для issues выбираются обращения, созданные в `[at-90 суток, at-7 суток]`; для каждого полностью наблюдается семисуточный срок. Ответ своевременен при первом ответе сопровождающего-человека не позднее `created_at+7 суток`. Отсутствие ответа входит в знаменатель. Самоответ автора и сообщение бота не считаются ответом сопровождения. Без подтверждённых ролей метрика unknown, а не приблизительный «ответ команды».

`Backlog(stuck,open)` = `100 * (1 - stuck/open)`. Выборка - открытые на at обращения, требующие действия команды. Исключаются подтверждённые состояния «ожидает автора» и «запланировано», по фиксированному отображению статусов/меток. Если отображение или полная выборка неизвестны, метрика unknown. stuck = более 30 суток после последнего содержательного действия команды; если его не было, после создания. Ровно 30 суток не считается превышением. Смена косметической метки и сообщения бота не обновляют это время.

При open=0 метрика insufficient. «Нет текущего завала» показывается как факт, но не превращается в идеальную поддержку.

`Issues(response,backlog)` объединяет показатели с весами 60/40. Окна и когорты разные намеренно: первая метрика оценивает входящие обращения, вторая - текущую очередь. Это не доля всех исторически закрытых задач. Время закрытия и число задач - справочные значения. Пример: 5 своевременных ответов из 10, stuck=0 при open=10 -> 70.

05 / Активность и сопровождение

`Activity(rhythm,response)` использует веса 60/40. Response рассчитывается так же, как в Issues, но только для предложений изменений; одни и те же обращения не учитываются в обеих категориях.

Ритм: за 365 суток до at адаптер выбирает уникальные UTC-дни с содержательными изменениями основной ветки (собственный код, документация, зависимости; исключены чисто сгенерированные файлы). Merge и исходный коммит одного изменения не дублируют событие. Бот не исключается автоматически, если его изменение содержательное. Между последовательными днями считаются законченные интервалы в сутках. Текущий незавершённый интервал в историю не включается. Требуются минимум 4 интервала (5 дней событий).

`Rhythm(intervals,gap)`:

```
Q75 = элемент отсортированной истории с индексом ceil(0,75*N)-1
b = max(30, Q75)
gap = (at - начало последнего UTC-дня события) / 24 часа
Rhythm = 100 * clamp((3*b-gap)/(2*b), 0, 1)
```

При gap<=b результат 100; при gap>=3*b - 0. Это отклонение от предыдущего ритма, не доказательство качества или заброшенности. Недостаточная история -> insufficient. Архивные/завершённые проекты маркируются отдельно; адаптер может передать not_applicable, но это не повышает полноту и не открывает доступ в полный рейтинг. Будущий отдельный профиль для таких проектов требует калибровки.

Лайки, количество коммитов, число авторов и возраст релиза не добавляют баллов. Не заявляется полная защита от имитации содержательных изменений. Смена режима сопровождения остаётся ограничением этого кандидата.

Пример ритма

Исторические интервалы: 10, 20, 30, 40 суток. Q75 = 30; b = 30. При текущем интервале 30 суток оценка ритма 100; при 60 сутках - 50; при 90 сутках - 0.

06 / Состояние кода

Функция `Code(complexLines,functionLines,duplicateLines,authoredLines)` использует собственные исполняемые строки без комментариев/пустых строк, vendor, generated, minified, lock-файлов, бинарных данных и фикстур, явно сгенерированных инструментами. Собственные тесты включаются; их наличие не означает покрытие тестами.

Анализатор должен полностью поддерживать синтаксис анализируемого кода и иметь закреплённую версию/конфигурацию. При неизвестном языке, усечённом дереве или ошибке парсинга нельзя отдавать полную категорию по одному поддержанному файлу. Нормализатор возвращает incomplete/not_applicable. rc1 не реализует анализаторы языков.

Структурная простота: `C=100*(1-complexLines/functionLines)`. complexLines - уникальные строки функций с цикломатической сложностью >15 по закреплённому анализатору; порог является параметром кандидата. Строки вложенных функций учитываются один раз, по самой внутренней функции. Минимум functionLines=100.

Дублирование: `D=100*max(0,1-(duplicateLines/authoredLines)/0,20)`. duplicateLines - объединение строк, участвующих в точных повторениях последовательностей минимум 50 лексических токенов и минимум 3 строк; пробелы и комментарии игнорируются, идентификаторы и литералы не переименовываются. Перекрывающиеся повторения не умножают число строк. Минимум authoredLines=100.

`CodeHealth = Combine(C,D; 65,35)`. При нехватке функции/строк её вес остаётся неизвестным. Например, известное только дублирование даёт coverage=0,35 и не представляет полную категорию. Нормализатор агрегирует счётчики по собственным модулям и хранит разбор по модулям для отображения слабых участков.

TODO/FIXME и их возраст можно показывать отдельно; их вклад в Score rc1 равен нулю. Добавление/удаление только комментариев при неизменном исполняемом коде не меняет входы Code. Метрика отражает структурную сопровождаемость, не все виды технического долга. Плотность остаётся чувствительной к добавлению простого кода: это проверяемое ограничение, а не гарантированная защита от накрутки.

Пример

100 строк в сложных функциях из 1000 анализируемых строк функций: C = 90. Дублируются 40 строк из 1000: D = 80. Code Health = 0,65 × 90 + 0,35 × 80 = 86,5. Комментарии TODO не изменяют этот балл.

07 / Security

Используются только отчёты SourceCraft AppSec. Known допустим, когда все применимые проверки SAST/SCA/secrets завершены, выборка находок полна, проверки относятся к текущей выбранной ревизии, и возраст каждого результата <=7 суток. Точное отображение статусов, применимости сканеров, идентификаторов и severity подтверждается спецификацией/живым API до интеграции. Параметр 7 суток - политика кандидата, не гарантия свежести базы уязвимостей.

Неизвестная severity, в том числе у секрета, не превращается автоматически в low или critical: ошибка нормализации, пока не согласовано отображение. Результат не гарантирует отсутствия вирусов. Known без находок означает только отсутствие проблем в указанном составе проверок.

Находка: непустой канонический ID, severity `critical/high/medium/low`, status `open/accepted/resolved/false_positive`. ID включает пространство имён анализатора и устойчивую идентичность находки. Одинаковые записи одного ID учитываются один раз. Противоречащие записи одного ID - ошибка. accepted не равно resolved. resolved и false_positive исключаются из текущих штрафов.

```
p(critical)=60; p(high)=25; p(medium)=8; p(low)=2
M = max(p), при пустом наборе 0
P = min(100, M + min(30, 0,25*(sum(p)-M)))
Security = 100-P
```

Максимальное накопление дополнительных штрафов - 30. При текущих весах самый большой штраф 90; не требуется, чтобы каждая категория использовала все возможные значения диапазона. Усиления за давность в rc1 нет: надёжность истории сначала нужно проверить.

При хотя бы одной актуальной critical со статусом open/accepted личный итог: `Score=min(Base,59)`, gate=`critical_appsec`. Это общий предел, а не cap категории. Значение 59 - проверяемое продуктовое решение кандидата.

Если ранее было ограничение, а актуальное состояние Security теперь неизвестно, итоговый личный `score=null`, gate=`security_status_unverified`. Предварительная база может отображаться только как база, не как улучшенный итог. Старое ограничение снимает только новый полный актуальный отчёт без таких critical. Историческую находку и её время сервис хранит отдельно. Исторический флаг не должен теряться при перезапуске или замене токена и проверяется в том же контексте репозитория/доступа. Пакет расчёта не реализует это хранилище.

Пример

Одна High и две Medium: P = 25 + 0,25 × (8 + 8) = 29; Security = 71. Одна открытая Critical: Security = 40, а итоговый личный Score не выше 59.

08 / Общий балл и рейтинг

Для любого набора компонентов: фиксированные ожидаемые веса w; известный score x и coverage c. Неизвестное имеет c=0.

```
K = sum(w*c)
coverage = K/sum(w)
observed_mean = sum(w*c*x)/K, если K>0
```

Поэтому частичная категория не получает полный вклад: известные 40% категории веса 20 дают только 8 единиц измеренного веса. Так обеспечивается согласованность расчёта по подпунктам и категориям.

Численный итог показывается только при общей полноте >=60% и минимум 3 категориях, каждая с coverage>=60%. Иначе статус insufficient, base/score=null. Полнота не умножается на балл. Показатель полноты не является вероятностью правильности.

Если `as_of-at >24 часа`, статус stale и base/score=null для текущего расчёта. Исторический отчёт сервис может показать отдельно с исходным временем. Ровно 24 часа допускается. Это политика свежести снимка; адаптер дополнительно проверяет свежесть каждого источника.

При достаточности Base=observed_mean. Затем применяется ограничение Security или удержание итогового балла при неподтверждённом старом ограничении. complete - coverage=1; иначе partial. В реализации используется допуск 1e-12 для погрешности float64, а не как способ скрыть реальные пропуски.

В рейтинг допускается только public + complete + актуальный снимок + подтверждённая публичная видимость. Неполные оценки остаются в каталоге без места. Сортировка: неокруглённый Score по убыванию, затем канонический ID репозитория по возрастанию. Профиль и версия одинаковы для сравниваемого рейтинга. Политика доступа может запретить публикацию даже при Rankable=true; этот флаг - необходимое, но не достаточное условие.

Не округляйте промежуточные значения. Для интерфейса - один десятичный знак; эталонные тесты сравнивают с допуском 1e-9. Скрытие плохой категории может поднять наблюдаемое среднее, но снимает место в рейтинге и отмечает неполноту. Нельзя утверждать, что сама формула полностью устраняет это явление.

09 / Рекомендации и проверка

Каждая рекомендация содержит metric_id, наблюдаемый факт, ссылку/идентификатор доказательства, действие, приоритет, профиль и версию. Сырые секреты не включаются. Доступ к доказательству проверяется отдельно.

Приоритеты rc1: текущая critical и неподтверждённое старое ограничение - critical (во втором случае действие: восстановить проверку, а не утверждать наличие новой проблемы); прочие измеренные категории <50 - high; от 50 включительно до 80 - medium; >=80 - без общей рекомендации. Неизвестный источник даёт отдельное информационное действие по получению данных, не совет исправлять выдуманный дефект. Внутри одинакового приоритета сортировка по фиксированному порядку категорий из таблицы, затем metric_id.

Конкретное действие должно опираться на подпункт: например, разобрать перечисленные failed CI, ответить на указанные обращения, добавить отсутствующие сведения. Общие шаблоны приоритетов не заменяют доказательств. Численную прибавку можно показывать только как условный пересчёт при неизменности остальных фактов и явном описании предполагаемых исправлений. Генератор рекомендаций и выдача доказательств ещё не интегрированы в эталонный агрегатор.

Что проверено

13 сценариев итогового расчёта прошли. Тесты проекта, Go vet и проверка синтаксиса интерфейса прошли. Покрытие нового пакета methodology - 93,0% операторов. Это покрытие кода, а не доказательство качества методики на реальных проектах.

Что остаётся сделать

Подтвердить поля API и права, реализовать нормализацию и анализаторы, обеспечить доступ к доказательствам, проверить новые, зрелые, крупные и архивные проекты, монорепозитории, чувствительность к весам. Автоматический детектор содержания документации и генератор рекомендаций пока не подключены.

Повторить проверку

```
go test ./internal/methodology
go run ./cmd/methodology-check
```

10 / Контрольные примеры

Синтетические данные: исходные оценки пяти публичных категорий равны 90. Время снимка фиксировано. Ни один пример не является результатом проверки реального репозитория.

Сценарий	Итог	Полнота	В рейтинг
Полный публичный отчёт	90	100%	Да
CI снижен до 20	72,5	100%	Да
Вместо CI нет доступа	90, частичный	75%	Нет
Известна только документация	Нет балла	20%	Нет
Известна половина Code Health	90, частичный	87,5%	Нет
Личный: Security недоступен	90, частичный	80%	Нет
Личный: открытая Critical	59 (база 80)	100%	Нет
Личный: принятый Critical-риск	59 (база 80)	100%	Нет
После ограничения потерян доступ	Итог удержан	80%	Нет
Новый полный чистый AppSec	92	100%	Нет
Приватные поля в публичном входе	90, без влияния	100%	Да
Снимку ровно 24 часа	90	100%	Да
Снимку больше 24 часов	Устарел, нет балла	100%	Нет

Исправленные контрпримеры

Issues: исходная формула давала 4030; корректная шкала нового примера даёт 70. Новый показатель измеряет ответы, а не долю закрытых задач.

TODO: добавление девяти новых отметок повышало прежний Code Health с 48 до 75. Теперь при неизменных структурных счётчиках результат остаётся 86,5.

Ограничение: cap только категории Security допускал общий Score 86. В rc1 критическая находка ограничивает итоговый личный балл до 59.

Основание документа

Спецификация METHODOLOGY_V2.md; результаты METHODOLOGY_V2_SCENARIOS.md; протокол METHODOLOGY_V2_VALIDATION.md. Файлы находятся в docs проекта Repo Health. Первичные материалы: ТЗ «8. Yandex Cloud.pdf», заметки созвона, «Метрики.pdf».